

# Redis Replacement Notes

---

## What Redis was doing in this project

Redis had two jobs in this app:

- Basket storage
- Response caching

### Basket storage

The shopping basket was not stored in PostgreSQL. Instead, the app stored each basket in Redis as a JSON string keyed by basket id.

Relevant code before the change:

- `API/Startup.cs` : created the Redis connection
- `Infrastructure/Data/BasketRepository.cs` : saved, loaded, and deleted baskets from Redis
- `API/Controllers/BasketController.cs` : used the basket repository

### Response caching

Product endpoints used Redis as a short-term cache.

Relevant code before the change:

- `Infrastructure/Services/ResponseCacheService.cs` : stored serialized API responses in Redis
- `API/Helpers/CachedAttribute.cs` : checked Redis before letting the controller run
- `API/Controllers/ProductsController.cs` : used `[Cached(600)]`

## Why replace Redis

For a portfolio deployment on Azure, managed Redis adds cost without adding much value for a low-traffic demo app.

## What replaced Redis

Redis was replaced with native .NET in-memory storage:

- `IMemoryCache` for API response caching
- `IMemoryCache` for basket storage

No external cache server is needed anymore.

## How the new version works

### Basket storage now

`Infrastructure/Data/BasketRepository.cs` now stores each basket in `IMemoryCache` for 30 days.

- when a basket is saved, it is serialized and put into app memory
- when a basket is requested, the app reads it from memory
- when a basket is deleted, the item is removed from memory

### Response caching now

`Infrastructure/Services/ResponseCacheService.cs` now stores serialized API responses in `IMemoryCache` for the requested TTL.

The `[Cached(600)]` attribute still works the same way from the controller's point of view.

## Code changes made

- `API/Startup.cs` : removed Redis connection setup and kept `AddMemoryCache()`
- `Infrastructure/Services/ResponseCacheService.cs` : replaced StackExchange Redis usage with `IMemoryCache`
- `Infrastructure/Data/BasketRepository.cs` : replaced Redis basket storage with `IMemoryCache`
- `API/Extensions/ApplicationServiceExtensions.cs` : changed `IBasketRepository` registration to singleton so basket data stays shared inside the running app
- `Infrastructure/Infrastructure.csproj` : removed the `StackExchange.Redis` package reference

## Tradeoffs of the new approach

Benefits:

- no Redis service to deploy
- lower Azure cost
- simpler setup
- minimal code changes

Limitations:

- baskets are lost if the app restarts
- cached responses are lost if the app restarts
- data is not shared across multiple app instances

## Final recommendation

For this repository on Azure with cost in mind: keep PostgreSQL, use in-memory basket storage, use in-memory response caching, and deploy a single app container.